Bézier Curve — Algebraic Expansion (Symbolic)

Explanation of the Program Output

Case 2: Quartic 3-D, 5 Control Points

`02-bezier-sympy.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains the output of `02-bezier-sympy.jl` when run with `CASE = 2`. It is a companion to the output explanation for `01-bezier-algebra.jl`. The curve and control points are identical — what changed is the computation engine. Script 01 used a hand-written `Poly` type with floating-point arithmetic. Script 02 uses **Symbolics.jl**, a computer algebra system, keeping all values as exact fractions throughout.

Contents

1 What Changed from Script 01 **3**

2 Header and Control Points **3**

3 Steps 1, 2, and 3 — Identical in Purpose **4**

4 Step 4 — Collecting Terms **5**
4.1 Understanding `(24//1)*(u^2)` 5
4.2 Collected Polynomials 6

5 Step 5 — Evaluation Table with Exact Fractions **6**
5.1 Two columns per axis 6
5.2 Verifying the exact fractions by hand 7
5.3 The z column 8

6 Verification Block **8**

7 Why Exact Arithmetic Matters **9**

1. What Changed from Script 01

Same curve. Same case. Same five steps. Different engine under the hood — and the difference shows up in every line of the output.

Script 01 (`01-bezier-algebra.jl`) built its own `Poly` type and stored polynomial coefficients as `Float64` numbers. All arithmetic was floating-point.

Script 02 (`02-bezier-sympy.jl`) replaces the hand-rolled `Poly` type entirely with **Symbolics.jl** — Julia's computer algebra system (CAS). Instead of numbers, the program manipulates *symbolic expressions* in the variable `u`. Coefficients are stored as `Rational{Int}` — exact integer fractions — so there is no floating-point rounding anywhere until the very last step when a decimal is explicitly requested.

The three visible differences in the output:

Location	Script 01	Script 02
Monomial coefficients	Plain integers: `24u^2`	Rational notation: `(24//1)*(u^2)`
Evaluation table	One column per axis (decimal only)	Two columns per axis: exact fraction + decimal
u values	Decimals: `0.25`, `0.50` ...	Exact fractions: `1//4`, `1//2` ...
Verification block	Two lines: expression + decimal	Three lines: expression, exact fraction, then decimal

Steps 1, 2, and 3 look almost identical between the two scripts — those steps are symbolic descriptions and the expansion results are the same polynomials. The real difference is visible in Steps 4 and 5.

2. Header and Control Points

```
====================================================================
  Bezier Curve - Algebraic Expansion  (via Symbolics.jl)
  Case 2  |  Quartic  |  3D  |  5 Control Points
====================================================================

  Control Points:
    P0 :  x = 0  y = 0  z = 1
    P1 :  x = 0  y = 4  z = 1
    P2 :  x = 4  y = 0  z = 1
    P3 :  x = 4  y = 4  z = 1
    P4 :  x = 5  y = 4  z = 1
```

The subtitle *via Symbolics.jl* is the only header difference from Script 01. The control

points are identical. Internally, however, these coordinates are stored as `Rational{Int}` (exact integer fractions) rather than `Float64` — so `4` here means exactly $\frac{4}{1}$, not the floating-point approximation 4.0.

This distinction matters later: every coefficient in every polynomial will be an exact fraction, produced entirely by integer arithmetic.

3. Steps 1, 2, and 3 — Identical in Purpose

```
STEP 1 - Bernstein Form
----------------------------------------------------------------
P(u)  =  [x(u)  y(u)  z(u)]

P(u)  =  B(4,0)*P0 + B(4,1)*P1 + ... + B(4,4)*P4

where:
  B(4,0,u)  =  (1-u)^4
  B(4,1,u)  =  4u(1-u)^3
  ...
```

```
STEP 2 - Substitution of Control Point Coordinates
----------------------------------------------------------------
x(u)  =  (1-u)^4*0  +  4u(1-u)^3*0  +  6u^2(1-u)^2*4  + ...
...
(Each basis term is now expanded by Symbolics.jl)
```

```
STEP 3 - Expand Each Bernstein Basis Term
----------------------------------------------------------------
B(4,0,u)  =  (1-u)^4
         =  1 - 4u + 6(u^2) - 4(u^3) + u^4

B(4,1,u)  =  4u(1-u)^3
         =  4u - 12(u^2) + 12(u^3) - 4(u^4)
...
```

These three steps are structurally identical to Script 01. The Bernstein formula, the substitution, and the expansion results are all the same polynomials.

One small visual difference in Step 3: Symbolics.jl wraps higher powers in parentheses — `6(u^2)` rather than `6u^2`. This is just a display choice by the CAS; mathematically the expressions are identical.

> **Step 2 closing line:** `(Each basis term is now expanded by Symbolics.jl)`
>
> In Script 01 the equivalent line said `(Each basis term will now be expanded individually)`, referring to the hand-written `expand_bernstein` function. Here it says `expanded by Symbolics.jl` — the CAS calls its own `expand()` function internally, which applies the binomial theorem and collects like terms in a single operation.

4. Step 4 — Collecting Terms

```
STEP 4 - Collect Terms - Final Monomial Polynomial
----------------------------------------------------------------
x(u) - weighted contributions:
  B(4,0)*   0  =  0
  B(4,1)*   0  =  0
  B(4,2)*   4  =  (24//1)*(u^2) - (48//1)*(u^3) + (24//1)*(u^4)
  B(4,3)*   4  =  (16//1)*(u^3) - (16//1)*(u^4)
  B(4,4)*   5  =  (5//1)*(u^4)
```

4.1. Understanding `(24//1)*(u^2)`

This notation is the most unfamiliar part of the output for anyone coming from Script 01. It needs to be read in two pieces.

The `//` operator — Julia's rational number literal. In Julia, `a//b` creates an *exact rational number* $\frac{a}{b}$ stored as a pair of integers. It never loses precision.

> `24//1` is the rational number $\frac{24}{1} = 24$, exactly.
>
> `1//4` is the rational number $\frac{1}{4} = 0.25$, exactly. Unlike `Float64(0.25)`, this is represented without any binary rounding — it is a genuine fraction in memory.
>
> `1//3` is $\frac{1}{3}$, exactly. `Float64(1/3)` would be `0.3333333333333333...` with rounding. `1//3` has no rounding at all.

Why does `24` become `24//1`? When Symbolics.jl expands an expression whose coefficients come from a `Rational{Int}` matrix, it preserves the rational type throughout. The coordinate `4` was stored as `4//1`; multiplying it by the integer coefficient `6` (from $\binom{4}{2} = 6$) gives `6 * (4//1) = 24//1`. Symbolics.jl then prints this rational coefficient as `(24//1)` to make the type explicit.

In short: `(24//1)*(u^2)` means exactly $24u^2$. The fraction notation is a transparency feature, not a complication.

> **Reading Step 4 contributions for $x(u)$:**
>
> | Symbolic output | Meaning | Script 01 equivalent |
> |---|---|---|
> | `(24//1)*(u^2) - (48//1)*(u^3) + (24//1)*(u^4)` | $24u^2 - 48u^3 + 24u^4$ | `24u^2 - 48u^3 +` |
> | `(16//1)*(u^3) - (16//1)*(u^4)` | $16u^3 - 16u^4$ | `16u^3 - 16u^4` |
> | `(5//1)*(u^4)` | $5u^4$ | `5u^4` |
>
> The numbers are identical to Script 01. Only the display format changed because the type changed.

4.2. Collected Polynomials

```
Collected:
  x(u)  =  (24//1)*(u^2) - (32//1)*(u^3) + (13//1)*(u^4)
  y(u)  =  (16//1)*u - (48//1)*(u^2) + (64//1)*(u^3) - (28//1)*(u^4)
  z(u)  =  1
```

Same final polynomials as Script 01:

$$x(u) = 24u^2 - 32u^3 + 13u^4$$

$$y(u) = 16u - 48u^2 + 64u^3 - 28u^4$$

$$z(u) = 1$$

The $z(u) = 1$ result is particularly clean — Symbolics.jl simply outputs `1` with no rational wrapper, because the sum of all Bernstein polynomials is identically 1 and the CAS recognises this and simplifies it completely.

5. Step 5 — Evaluation Table with Exact Fractions

This is the step that most visibly sets Script 02 apart.

```
      u  x(u) exact       decimal  y(u) exact       decimal  z(u) exact
      ↪  decimal

↪  ---------------------------------------------------------------------------------------
      0           0      0.000000           0      0.000000           1
      ↪  1.000000
  1//4    269//256      1.050781     121//64      1.890625           1
  ↪  1.000000
  1//2      45//16      2.812500        9//4      2.250000           1
  ↪  1.000000
  3//4   1053//256      4.113281     201//64      3.140625           1
  ↪  1.000000
      1           5      5.000000           4      4.000000           1
      ↪  1.000000
```

5.1. Two columns per axis

Each axis now has two columns: **exact** and **decimal**.

The **exact** column is the result of substituting the exact rational u value (e.g. `1//4`) into the symbolic polynomial and simplifying — the CAS returns an exact fraction.

The **decimal** column converts that fraction to `Float64` only at the very last moment, purely for human readability.

> **This is the core advantage of symbolic computation.**
>
> In Script 01, $x(0.25)$ was computed as:
>
> $$24 \times 0.0625 - 32 \times 0.015625 + 13 \times 0.00390625 = 1.05078125$$

using floating-point arithmetic at every multiplication.

In Script 02, $x\left(\frac{1}{4}\right)$ is computed as:

$$24 \times \frac{1}{16} - 32 \times \frac{1}{64} + 13 \times \frac{1}{256} = \frac{384}{256} - \frac{128}{256} + \frac{13}{256} = \frac{269}{256}$$

using exact integer fraction arithmetic. The decimal 1.050781 is then read off from $\frac{269}{256}$. No rounding occurred anywhere in the computation.

5.2. Verifying the exact fractions by hand

Verify $x\left(\frac{1}{4}\right) = \frac{269}{256}$:

Substitute $u = \frac{1}{4}$ into $x(u) = 24u^2 - 32u^3 + 13u^4$:

$$24 \cdot \left(\tfrac{1}{4}\right)^2 - 32 \cdot \left(\tfrac{1}{4}\right)^3 + 13 \cdot \left(\tfrac{1}{4}\right)^4 = \frac{24}{16} - \frac{32}{64} + \frac{13}{256}$$

Converting to a common denominator of 256:

$$= \frac{384}{256} - \frac{128}{256} + \frac{13}{256} = \frac{384 - 128 + 13}{256} = \frac{269}{256}$$

As decimal: $269 \div 256 = 1.05078125 \approx 1.050781$ ✓

Verify $y\left(\frac{1}{2}\right) = \frac{9}{4}$:

Substitute $u = \frac{1}{2}$ into $y(u) = 16u - 48u^2 + 64u^3 - 28u^4$:

$$16 \cdot \tfrac{1}{2} - 48 \cdot \tfrac{1}{4} + 64 \cdot \tfrac{1}{8} - 28 \cdot \tfrac{1}{16} = 8 - 12 + 8 - \tfrac{7}{4} = 4 - \tfrac{7}{4} = \tfrac{9}{4}$$

As decimal: $9 \div 4 = 2.25$ ✓

Verify $x(1) = 5$ and $y(1) = 4$:

$$x(1) = 24 - 32 + 13 = 5 ✓$$
$$y(1) = 16 - 48 + 64 - 28 = 4 ✓$$

At the endpoints the exact values are whole integers — no fractions needed. This confirms the curve starts at $P_0 = (0, 0, 1)$ and ends at $P_4 = (5, 4, 1)$.

5.3. The z column

Every entry in the z column is exactly `1` with decimal `1.000000`. This is the partition-of-unity property in action: all five z-coordinates are 1, so $z(u) = 1$ for all u, and substituting any rational u into the constant polynomial $z(u) = 1$ gives exactly `1` — no fraction, no decimal needed.

6. Verification Block

```
Verification at u = 1//2:
  x(1//2)  =  (24//1)*(u^2) - (32//1)*(u^3) + (13//1)*(u^4)
           =  45//16
           =  45//16  ~  2.812500

  y(1//2)  =  (16//1)*u - (48//1)*(u^2) + (64//1)*(u^3) - (28//1)*(u^4)
           =  9//4
           =  9//4  ~  2.250000

  z(1//2)  =  1
           =  1
           =  1  ~  1.000000
```

The verification block now has **three lines** per axis, compared to Script 01's two.

Line	Content	Meaning
1	Symbolic expression	The full polynomial as Symbolics.jl stored it
2	Exact rational result	The CAS substituted $u = \frac{1}{2}$ and simplified to a fraction
3	Fraction ≈ decimal	The same fraction converted to `Float64` for readability

Tracing $x\left(\frac{1}{2}\right)$ through the three lines:

Line 1 — the stored symbolic expression:

$$(24//1) \cdot u^2 - (32//1) \cdot u^3 + (13//1) \cdot u^4 \quad \equiv \quad 24u^2 - 32u^3 + 13u^4$$

Line 2 — substitute $u = \frac{1}{2}$ and simplify:

$$24 \cdot \tfrac{1}{4} - 32 \cdot \tfrac{1}{8} + 13 \cdot \tfrac{1}{16} = 6 - 4 + \tfrac{13}{16} = 2 + \tfrac{13}{16} = \tfrac{45}{16}$$

So `45//16` is correct. ✓

Line 3 — convert to decimal: $45 \div 16 = 2.8125$, displayed as `2.812500`. ✓

Why show the fraction twice (lines 2 and 3)?

Line 2 is the CAS result as returned by `substitute()` — it is still a Symbolics expression, printed as `45//16`. Line 3 is after calling `to_exact_rational()` to extract the plain Julia `Rational{Int}` value, and then `Float64()` for the decimal. The two-step display confirms that the CAS result and the extracted rational agree before the decimal conversion.

7. Why Exact Arithmetic Matters

You might wonder: both scripts give the same curve. So why bother with the extra complexity of symbolic computation?

The answer is trust and precision.

With Script 01 (floating-point), the decimal `1.050781` could in principle be a rounded approximation of some irrational number — you cannot tell just from the decimal alone.

With Script 02 (exact), the result `269//256` is a *proof*: the exact value of $x\left(\frac{1}{4}\right)$ on this curve is the fraction $\frac{269}{256}$. No rounding ever occurred. You can verify it with pencil and paper.

For a quartic curve with integer control points and rational u values, the output is always a rational number. Script 02 *finds and displays* that rational number exactly. Script 01 *approximates* it with a float.

Both are correct for practical use. But Script 02 gives you something Script 01 cannot: a result you can algebraically verify without worrying about floating-point error.

A note on the `(24//1)` display.

You might wish Symbolics.jl printed `24` instead of `(24//1)`. That is a reasonable aesthetic preference. The fraction notation is the CAS being explicit about types — it is telling you these are rational coefficients, not floating-point approximations. Once you have read `24//1` as "exactly 24" a few times, it stops feeling strange.

Like meeting someone with an unusual name. After the third time you just call them by it without thinking.

Output explanation for `02-bezier-sympy.jl`, Case 2 | Code by E.R. Nurwijayadi | Explanations prompt-directed by E.R. Nurwijayadi, AI-composed